PostgreSQL 11 → 16 Migration Runbook

Standalone PG11 (WebFOCUS repository) → the db01 / db02 PG16 streaming-replication cluster
Globals + database (excluding **public.botldata** data) + size-filtered **botldata** reload

Field	Value
Source	`PostgreSQL 11 standalone, single database (WebFOCUS repo)`
Target primary	`db01 (PostgreSQL 16)`
Target standby	`db02 (streams via slot ssncpri — do not restore here)`
Table handled separately	`public.botldata (column: report)`
Row filter	`octet_length(report) < 20*1024*1024 (rows under 20 MB)`
FK	`public.botldata(reportid) → public.botlib(reportid)`
Client binaries	`/usr/pgsql-16/bin (PG16 tools, used against PG11 too)`
Staging directory	`/apps/backups/pg11to16`
To fill in	`<SRC_HOST> = PG11 host/IP · <SRC_DB> = repo DB name`

0 Scenario & design decisions

- **Dump with the PG16 binaries, not PG11.** A newer pg_dump reading an older server is the supported direction; the reverse is not. Every command uses `/usr/pgsql-16/bin`.
- **--exclude-table-data, not --exclude-table.** The botldata definition, PK, indexes and its FK stay in the main dump in correct dependency order; only its rows are omitted, then reloaded filtered. This sidesteps the dependency-ordering traps of fully excluding the table.
- **Row filtering needs \copy, not pg_dump.** pg_dump has no row-level WHERE in any version, so the <20 MB slice is exported with `\copy (SELECT … WHERE …)`.
- **Restore on the primary only.** db01 receives everything; db02 replays it through the ssncpri slot automatically.

> **Why PG16 tools against a PG11 source**
> pg_dump / pg_dumpall / pg_restore are backward-compatible downward: v16 can read a v11 server and produce archives v16 restores cleanly. Running v11 tools into a v16 cluster is unsupported. Point the v16 client at the source with `-h <SRC_HOST>`.

1 Pre-flight checks

Run these on the source before backing up. Use a `~/.pgpass` (chmod 600) or `PGPASSWORD` so nothing prompts.

1.1 Confirm the PG16 client is reachable and the source answers

```
/usr/pgsql-16/bin/pg_dump --version
/usr/pgsql-16/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  -c "SELECT version();"
```

1.2 Drop amcheck on the source so it is not carried into the dump

```
/usr/pgsql-16/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  -c "DROP EXTENSION IF EXISTS amcheck;"
```

1.3 Eyeball remaining extensions and legacy WITH OIDS tables

```
/usr/pgsql-16/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> -c "\dx"
# WITH OIDS was removed in PG12 -- this should return zero rows:
/usr/pgsql-16/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  -c "SELECT relname FROM pg_class WHERE relhasoids;"
```

1.4 FK sanity check — confirm nothing references botldata

```
/usr/pgsql-16/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> -c \
"SELECT conrelid::regclass FROM pg_constraint \
 WHERE contype='f' AND confrelid='public.botldata'::regclass;"
```

> **Expected: zero rows**
> botldata is the **child** (it points to botlib), so nothing downstream should depend on it. Zero rows means the filtered reload is safe and no sectioned restore is needed. If rows come back, stop — see §2.

> **Capacity before you start**
> The <20 MB botldata slice is written as an uncompressed CSV — make sure `/apps/backups` has room. The restore also generates a lot of WAL; given this pair once drove slot ssncpri to `lost`, watch archive/disk headroom and db02 replay lag throughout.

2 Referential-integrity analysis (why the filter is safe)

The only constraint on the table is `public.botldata(reportid) → public.botlib(reportid)`, so botldata is the child and botlib the parent. Dropping the >20 MB botldata rows removes child rows only:

- botlib (parent) is migrated in full by the main dump, so every surviving botldata row still points to a valid parent — no orphans.
- In the single pg_restore pass the FK is (re)created during post-data while botldata is still empty (trivially valid); the filtered rows load afterward and are validated one-by-one against the fully-loaded botlib.
- Nothing references botldata, so discarding the large rows breaks nothing downstream.

> **If §1.4 returned any table**
> Then some table *is* a child of botldata and filtering its rows would orphan those children. Do not run the simple flow: switch to a sectioned restore (`--section=pre-data` → `--section=data` → load botldata → `--section=post-data`) and decide how to handle the orphaned children first.

3 Source-side backups

3.0 Staging directory

```
mkdir -p /apps/backups/pg11to16
```

3.1 Globals first (roles, tablespaces)

```
/usr/pgsql-16/bin/pg_dumpall -h <SRC_HOST> -p 5432 -U postgres \
  --globals-only -f /apps/backups/pg11to16/01_globals.sql
```

3.2 Database dump — botldata structure kept, its data excluded

```
/usr/pgsql-16/bin/pg_dump -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  --create \
  --exclude-table-data=public.botldata \
  -Fc -v \
  -f /apps/backups/pg11to16/02_maindb_no_botldata_data.dump
```

For a large repo you can swap `-Fc` for `-Fd -j 4` (directory format, parallel dump).

3.3 The botldata rows under 20 MB on the report column

```
/usr/pgsql-16/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> -c \
"\copy (SELECT * FROM public.botldata \
 WHERE octet_length(report) < 20*1024*1024) \
 TO '/apps/backups/pg11to16/03_botldata_le20mb.csv' WITH (FORMAT csv)"
```

> **octet_length assumes report is bytea / text**
> It counts stored bytes — exactly the 20 MB test you want. If report is actually a large object (lo / oid), \copy would export only the OID, not the content; that needs a different approach.

4 Transfer to the primary

Skip this if you are running the dumps *from* db01 — the files are already local.

```
scp -r /apps/backups/pg11to16 postgres@db01:/apps/backups/
```

5 Restore on the primary (db01 only)

> **Never restore on db02**
> All three restore steps target db01. The standby db02 replays the changes through the ssncpri slot on its own — touching it directly will break replication.

5.1 Globals

```
/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d postgres \
  -f /apps/backups/pg11to16/01_globals.sql
```

"role / tablespace already exists" errors here are harmless.

5.2 Database (creates the DB, botldata present but empty)

```
/usr/pgsql-16/bin/pg_restore -h db01 -p 5432 -U postgres -d postgres \
  --create -Fc -v -j 4 \
  /apps/backups/pg11to16/02_maindb_no_botldata_data.dump
```

5.3 Load the filtered botldata rows

```
/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d <SRC_DB> -c \
"\copy public.botldata \
 FROM '/apps/backups/pg11to16/03_botldata_le20mb.csv' WITH (FORMAT csv)"
```

6 Post-restore validation

6.1 Refresh planner statistics (none exist until analyzed)

```
/usr/pgsql-16/bin/vacuumdb -h db01 -p 5432 -U postgres -d <SRC_DB> \
  --analyze-in-stages
```

6.2 Verify row counts (botldata should be smaller than source)

```
/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d <SRC_DB> -c \
"SELECT count(*) FROM public.botldata;"
```

6.3 Replication health

```
# on the primary: is db02 streaming and caught up?
/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d postgres -c \
"SELECT client_addr, state, sync_state, replay_lag FROM pg_stat_replication;"

# on the standby: should return t, counts should match the primary
/usr/pgsql-16/bin/psql -h db02 -p 5432 -U postgres -d postgres -c \
"SELECT pg_is_in_recovery();"
```

7 Rollback / re-run

- The target is a fresh restore, so re-running is clean: on db01 drop the half-restored DB (`DROP DATABASE <SRC_DB>;` from the postgres DB, after disconnecting sessions) and repeat from §5. The source is untouched throughout.
- If only the botldata load failed, you do not need to redo §5.2 — `TRUNCATE public.botldata;` on db01 and re-run §5.3.
- Keep the three artifacts in /apps/backups/pg11to16 until db02 shows the matching count and zero lag.

Appendix A Full sequence (quick copy)

Every long command is wrapped with backslash continuations, so each block still pastes and runs as a single command.

Source

```
mkdir -p /apps/backups/pg11to16

/usr/pgsql-16/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  -c "DROP EXTENSION IF EXISTS amcheck;"

/usr/pgsql-16/bin/pg_dumpall -h <SRC_HOST> -p 5432 -U postgres \
  --globals-only -f /apps/backups/pg11to16/01_globals.sql

/usr/pgsql-16/bin/pg_dump -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> \
  --create --exclude-table-data=public.botldata -Fc -v \
  -f /apps/backups/pg11to16/02_maindb_no_botldata_data.dump

/usr/pgsql-16/bin/psql -h <SRC_HOST> -p 5432 -U postgres -d <SRC_DB> -c \
"\copy (SELECT * FROM public.botldata \
 WHERE octet_length(report) < 20*1024*1024) \
 TO '/apps/backups/pg11to16/03_botldata_le20mb.csv' WITH (FORMAT csv)"

scp -r /apps/backups/pg11to16 postgres@db01:/apps/backups/
```

Primary (db01)

```
/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d postgres \
  -f /apps/backups/pg11to16/01_globals.sql

/usr/pgsql-16/bin/pg_restore -h db01 -p 5432 -U postgres -d postgres \
  --create -Fc -v -j 4 \
  /apps/backups/pg11to16/02_maindb_no_botldata_data.dump

/usr/pgsql-16/bin/psql -h db01 -p 5432 -U postgres -d <SRC_DB> -c \
"\copy public.botldata \
 FROM '/apps/backups/pg11to16/03_botldata_le20mb.csv' WITH (FORMAT csv)"

/usr/pgsql-16/bin/vacuumdb -h db01 -p 5432 -U postgres -d <SRC_DB> \
  --analyze-in-stages
```